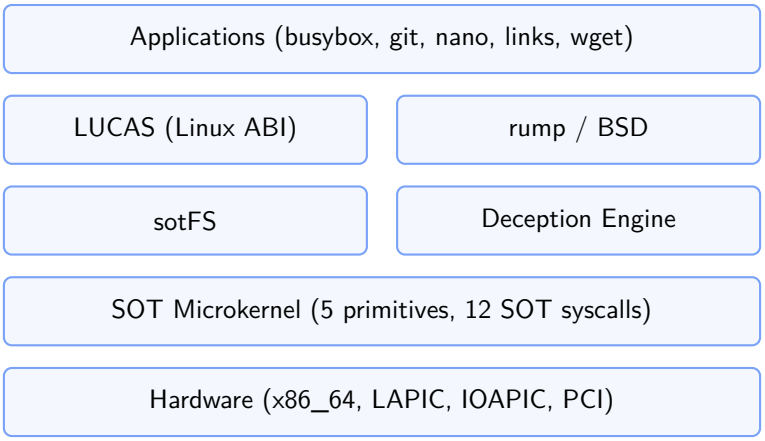
sotX System Manual

The SOT Exokernel, sotX Microkernel, and sotFS Filesystem

Version 0.2.0 — Codename: Project STYX



Lucas Sotomayor

2026-04-10

Contents

List of Figures

List of Tables

Listings

Part I

Architecture

Chapter 1

Introduction

1.1 What is sotX?

`sotX` is a verified microkernel operating system written from scratch in Rust, designed around five core principles: minimal kernel surface, capability-based security, transactional secure objects, deception-aware personality layers, and formal verifiability. The system runs on the `x86_64-unknown-none` bare-metal target and boots via the Limine v8 BIOS protocol.

The name reflects two layers: the SOT (Secure Object Transaction) exokernel that provides the fundamental abstraction for all kernel objects, and the BSD personality layer that presents a POSIX-compatible interface to userspace programs. In practice, the system also hosts a full Linux ABI compatibility layer called LUCAS (Linux User-mode Compatibility And Syscall translation), allowing real musl-static and glibc-linked binaries to run unmodified.

1.2 Design Philosophy

The architecture follows a strict separation of concerns:

1. **Minimal kernel.** The microkernel implements exactly five primitives: scheduling, IPC, capabilities, frame allocation, and IRQ virtualization. Everything else—virtual memory, filesystems, drivers, networking—lives in userspace servers.
2. **Capabilities as the sole access mechanism.** Every kernel object is referenced through a capability. There are no ambient authorities, no root user, no global namespaces.
3. **Transactional objects.** The SOT exokernel wraps kernel objects in transactional envelopes with three consistency tiers, enabling atomic multi-object operations with provenance tracking.
4. **Deception as a first-class primitive.** Capability interposition enables transparent rewriting of system call results, allowing the OS to present fabricated views of its own identity to adversaries.
5. **Formal foundations.** Six TLA+ specifications cover the core subsystems, and the sotFS filesystem is verified through DPO rewriting rules with bounded treewidth guarantees.

1.3 Document Organization

This manual is organized into five parts:

Part I: Architecture (Chapters **??**–**??**) covers the system-level design, boot process, memory layout, and the SOT exokernel model.

Part II: Subsystems (Chapters **??**–**??**) details the capability system, IPC mechanisms, scheduler, and security infrastructure.

Part III: Personalities (Chapters **??**–**??**) describes the Linux ABI compatibility layer, the BSD/rump personality, and the deception engine.

Part IV: Services & Drivers (Chapters **??**–**??**) covers the sotFS filesystem, device drivers, and the networking stack.

Part V: Operations (Chapters **??**–**??**) provides build instructions, debugging guides, a complete syscall reference, and formal verification details.

1.4 Conventions Used in This Manual

- System call names are set in `monospace`.
- Register names use the `rdi`, `rsi`, `rdx` convention.
- Hexadecimal addresses are prefixed: `0xB00000`.
- Code listings use the Rust language unless otherwise noted.
- The symbol ✓ indicates a feature that has been verified end-to-end in the boot test pipeline.

Chapter 2

System Architecture

2.1 Architectural Overview

`sotX` is structured as a layered system with a minimal microkernel at its core. Figure **??** shows the complete architecture stack from hardware to applications.

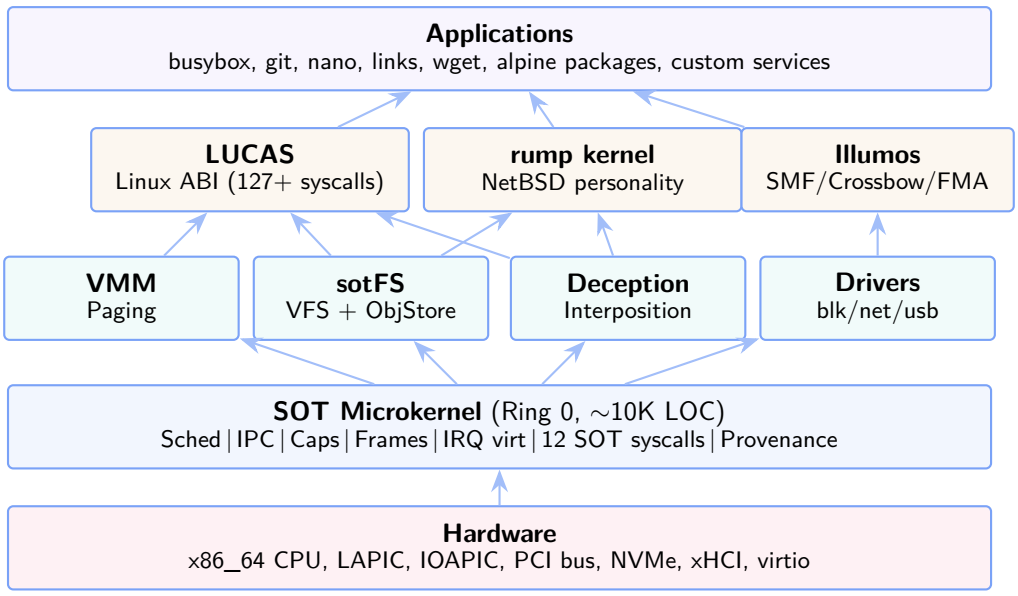


Figure 2.1: sotX architecture stack. The microkernel provides five primitives; everything above runs in Ring 3 as capability-holding services.

2.2 The Five Kernel Primitives

The microkernel implements exactly five primitives. All other OS functionality is built on top of these through userspace services communicating via IPC.

Scheduling. A four-level priority Multi-Level Queue (MLQ) scheduler with per-core Chase-Lev work-stealing deques. Priority levels are: *realtime* (0), *high* (1), *normal* (2), and *low* (3). The scheduler supports up to 16 CPUs via SMP with LAPIC timer-driven preemption and IPI-based cross-core wakeup.

IPC. Two mechanisms: synchronous endpoints (L4-style rendezvous where sender blocks until receiver calls) and asynchronous channels (lock-free SPSC ring buffers for high-throughput

data transfer). IPC also supports capability transfer in the message registers.

Capabilities. All kernel objects are accessed through capabilities. A Capability Derivation Tree (CDT) tracks parent-child relationships for delegation and epoch-based O(1) revocation. A rights bitmask (Read/Write/eXecute/Grant/Revoke) provides monotonic attenuation.

Frame Allocation. A bitmap-based physical frame allocator manages all physical memory. A slab allocator built on top provides efficient allocation for fixed-size kernel objects (thread control blocks, capability entries, endpoint structures).

IRQ Virtualization. Hardware interrupts are demultiplexed by the kernel and delivered as IPC messages to registered userspace handlers. This allows device drivers to run entirely in Ring 3 without any kernel-mode driver code.

2.3 The SOT Exokernel Layer

On top of the five primitives, the SOT (Secure Object Transaction) layer provides 12 additional syscalls (numbers 300–311) that implement:

- **Secure Objects (SO):** opaque, typed blobs that serve as the universal container for all named kernel resources.
- **Transactions:** three tiers of consistency—Tier 0 (RCU read-only snapshots), Tier 1 (WAL single-object writes), and Tier 2 (two-phase commit for multi-object atomic operations).
- **Provenance:** per-CPU SPSC ring buffers (4096 entries each) that record a SHA-256 hash-linked DAG of all SO mutations, enabling full audit trails and tamper detection.

2.3.1 Category-Theoretic Foundations

The SOT model is grounded in five categories with three functors:
Definition 2.1 (SOT Categories)**.** The system defines five categories:

$$\mathbf{SO} : \text{Secure Objects (morphisms = capability derivations)} \tag{2.1}$$
$$\mathbf{Dom} : \text{Protection Domains (morphisms = domain transitions)} \tag{2.2}$$
$$\mathbf{Chan} : \text{Channels (morphisms = message flows)} \tag{2.3}$$
$$\mathbf{Tx} : \text{Transactions (morphisms = commit/abort)} \tag{2.4}$$
$$\mathbf{Prov} : \text{Provenance (morphisms = hash-chain links)} \tag{2.5}$$

Three functors connect these categories:

- $F_{\text{Access}} : \mathbf{Dom} \to \mathbf{SO}$ maps domain transitions to capability derivations.
- $F_{\text{Prov}} : \mathbf{Tx} \to \mathbf{Prov}$ maps transaction commits to provenance entries.
- $F_{\text{Deception}} : \mathbf{SO} \to \mathbf{SO}$ is an endofunctor that transparently rewrites SO contents for deception.

2.4 The 12 SOT Syscalls

Table **??** lists the complete SOT syscall interface.

Table 2.1: SOT exokernel syscalls (numbers 300–311).

#	Name	Args	Description
300	`so_create`	type, size, data	Create a new Secure Object
301	`so_read`	so_cap, off, len	Read bytes from an SO
302	`so_write`	so_cap, off, data	Write bytes to an SO
303	`so_destroy`	so_cap	Destroy an SO and free storage
304	`cap_derive`	src_cap, rights	Derive child cap with restricted rights
305	`cap_revoke`	cap	Revoke cap and all descendants in CDT
306	`chan_send`	chan_cap, msg	Send message on async channel
307	`chan_recv`	chan_cap	Receive message from async channel
308	`domain_enter`	dom_cap	Enter a protection domain
309	`domain_yield`		Yield back to calling domain
310	`tx_commit`	tx_handle	Commit a transaction (Tier 1 or 2)
311	`tx_abort`	tx_handle	Abort a transaction, discard writes

Chapter 3

Boot Process

3.1 Boot Sequence Overview

The system boots through four distinct phases, illustrated in Figure ??.

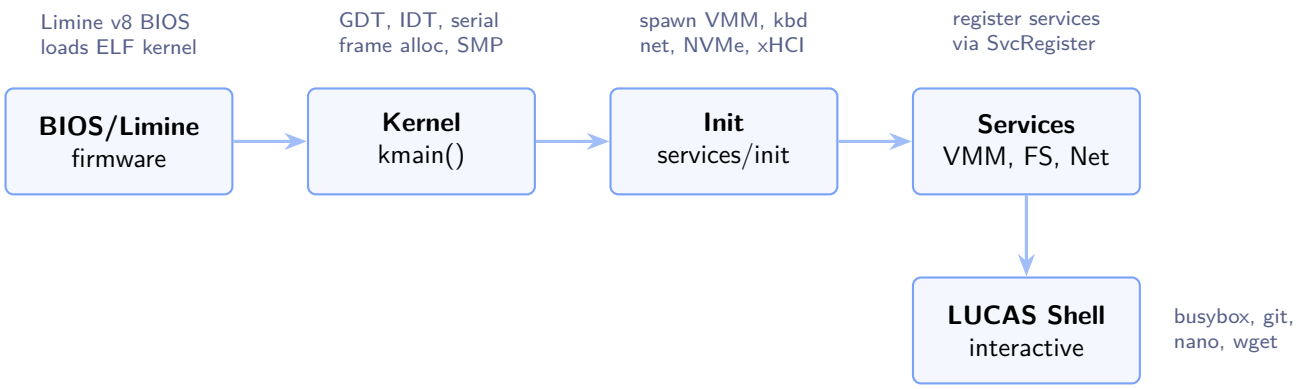


Figure 3.1: Boot sequence from firmware through kernel initialization to the interactive LUCAS shell.

3.1.1 Phase 1: Firmware and Bootloader

The Limine v8 BIOS bootloader loads the kernel ELF binary from a GPT+FAT32 disk image. The kernel must be compiled as `ET_EXEC` (not `ET_DYN`), which requires the `-C relocation-model=static` flag. Limine passes control to `kmain()` with a set of boot information structures describing available memory, framebuffer, and the CPIO initrd archive.

The boot configuration lives in `boot/limine.conf`:

```
serial: yes
timeout: 0

/sotX
  protocol: limine
  kernel_path: boot():/sotos-kernel
  module_path: boot():/initrd.cpio
```

Listing 3.1: Limine boot configuration (excerpt).

3.1.2 Phase 2: Kernel Initialization

The kernel initialization sequence in `kmain()` follows a strict order:

1. **Serial FIRST.** Initialize COM1 serial output before any assertions or debug prints. This is a critical invariant—panics before serial is initialized produce no diagnostics.
2. **GDT and IDT.** Load the Global Descriptor Table and Interrupt Descriptor Table. Configure IST entries: #DF (index 0), #GP (index 2), misc (index 3). *Note:* #PF does not use IST to avoid corruption when multiple threads fault simultaneously.
3. **Frame allocator.** Initialize the bitmap-based physical frame allocator from the Limine memory map.
4. **Slab allocator.** Initialize the kernel slab allocator on top of the frame allocator.
5. **Capability tables.** Initialize the global capability table and CDT structures.
6. **Scheduler.** Initialize per-CPU run queues and the Chase-Lev work-stealing deques.
7. **SMP.** Wake Application Processors (APs) via INIT-SIPI-SIPI. Enable SSE on all CPUs (CR0.EM=0, CR4.OSFXSR, CR4.OSXMMEXCPT).
8. **LAPIC timer.** Calibrate and start the Local APIC timer for preemptive scheduling.
9. **Init process.** Load the `init` service from the initrd CPIO archive and create the first userspace thread.

3.1.3 Phase 3: Init Service

The `services/init` process is the first userspace program. It reads BootInfo from `0xB00000` to obtain root capabilities, then spawns the core services:

- **VMM server:** manages page tables and handles #PF (page faults) for all user processes.
- **Keyboard driver:** reads PS/2 scan codes and distributes input events via a shared ring buffer at `0x510000`.
- **Network service:** implements the IP/TCP/UDP/ICMP stack.
- **NVMe driver:** communicates with NVMe SSDs via PRP lists and IPC.
- **xHCI driver:** manages USB host controllers and HID devices.

Each service registers itself with the kernel's service registry via `SvcRegister` (syscall 130), making its endpoint discoverable by other services through `SvcLookup` (syscall 131).

3.1.4 Phase 4: Shell and Applications

After services are running, init launches the LUCAS shell, which provides a POSIX-like command-line interface supporting pipes (up to 10 commands), background jobs, environment variables, and scripting. Real Linux binaries such as `busybox`, `git`, `nano`, and `links` run through the Linux ABI compatibility layer.

Chapter 4

Memory Architecture

4.1 Physical Memory Management

Physical memory is managed by a bitmap-based frame allocator. Each bit in the bitmap represents a single 4 KiB frame. The allocator is initialized from the Limine memory map, which marks usable regions. A spinlock (`spin::Mutex`) protects concurrent access.

The slab allocator provides efficient allocation for fixed-size kernel objects, reducing fragmentation for frequently allocated structures such as thread control blocks and capability entries.

4.2 Virtual Address Space Layout

Each process has its own address space (CR3). The init process uses the layout shown in Table **??**.

Table 4.1: Init process virtual address space layout.

Address	Size	Content
`0x200000`	variable	ELF `.text` (kernel or service code)
`0x510000`	4 KiB	Keyboard ring buffer
`0x520000`	4 KiB	Mouse ring buffer
`0x900000`	16 KiB	Stack (4 pages)
`0xA00000`	4 KiB	SPSC ring (IPC)
`0xB00000`	4 KiB	BootInfo page (root caps)
`0xB80000`	4 KiB	vDSO page (R+X)
`0xC00000+`	variable	Virtio/MMIO device regions (UC mapped)
`0xD00000+`	variable	ObjectStore + VFS data
`0xE30000+`	variable	Child process stacks
`0x2000000`	1 MiB	`brk` heap
`0x3000000`	variable	`mmap` region
`0x6000000`	variable	`INTERP_LOAD_BASE` (dynamic linker)

4.3 Copy-on-Write Fork

The `SYS_AS_CLONE` (syscall 125) implements real copy-on-write fork:

1. The `clone_cow()` function walks all user-space PTEs in the parent's page tables.
2. Both parent and child PTEs are marked read-only.

3. A per-frame reference count (`FRAME_REFCOUNT`) is incremented.
4. On write fault (error code `& 0x03 == 0x03`), the VMM allocates a new frame, copies the data via HHDM (Higher Half Direct Map), and remaps the page as writable.

A critical implementation detail: #PF (page fault) must *not* use an IST (Interrupt Stack Table) entry, because shared IST stacks corrupt when multiple threads fault simultaneously during CoW fork. Instead, #PF uses the per-thread kernel stack (TSS RSP0).

4.4 Guard Pages and W^X

Every thread stack is bounded by guard pages—unmapped pages that trigger immediate #PF on stack overflow rather than silently corrupting adjacent memory. The kernel enforces W^X (Write XOR Execute): no page may be simultaneously writable and executable. The `SYS_PROTECT` (syscall 134) provides `mprotect`-like functionality with W^X enforcement.

Part II

Subsystems

Chapter 5

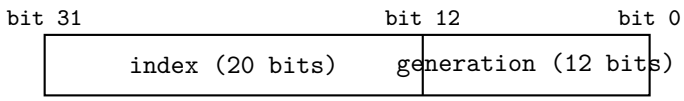
Capability System

5.1 Capability Model

Every kernel object is accessed through a capability. A capability is a protected reference that bundles an object identity with an access rights mask. There are no ambient authorities in the system—the only way to perform any operation is to present a valid capability.

5.1.1 CapId Format

A `CapId` is a 32-bit value composed of two fields:



The 20-bit index allows up to $2^{20} = 1{,}048{,}576$ simultaneous capabilities. The 12-bit generation counter prevents use-after-free attacks: when a capability is revoked, the generation is bumped, and any stale `CapId` with the old generation is rejected.

5.1.2 Rights Bitmask

Each capability carries a 5-bit rights mask:

Bit	Name	Meaning
0	Read	Read object contents
1	Write	Modify object contents
2	Execute	Execute/invoke the object
3	Grant	Derive child capabilities
4	Revoke	Revoke child capabilities

Rights are *monotonically attenuated*: a derived capability can only have a subset of its parent's rights. The `Rights::restrict()` method enforces this invariant.

5.2 Capability Derivation Tree

The CDT (Capability Derivation Tree) tracks the parent-child relationship between capabilities. Each capability entry stores a parent pointer. When a capability is revoked via `cap_revoke`, the

revocation propagates to all descendants through an epoch-based mechanism that achieves O(1) amortized cost.

Figure ?? shows a typical derivation tree with progressive attenuation.

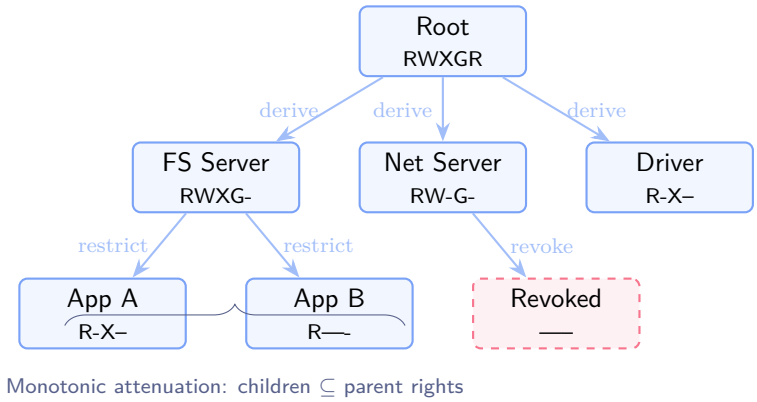


Figure 5.1: Capability Derivation Tree with monotonic attenuation. The dashed node shows a revoked capability—all descendants are also invalidated.

5.3 Interposition Policies

The capability system supports four interposition policies that control how operations on a capability are handled:

Passthrough. The operation proceeds directly to the underlying object with no modification.

Inspect. The operation is logged to the provenance subsystem before proceeding.

Redirect. The operation is transparently forwarded to a different object (used by the deception engine).

Fabricate. The result is synthesized without touching the real object (used for honeypot responses).

5.4 Capability Derive and Revoke Example

Listing ?? shows how a service derives a restricted capability and later revokes it.

```
use sotos_common::sys;
use sotos_common::cap::{CapId, Rights};

// Derive a read-only capability from our full-access cap
let full_cap: CapId = my_resource_cap;
let ro_rights = Rights::READ;
let ro_cap = sys::cap_derive(full_cap, ro_rights)
    .expect("cap_derive failed");

// Grant the read-only cap to a client via IPC
let mut msg = Message::new();
msg.set_cap(0, ro_cap);
sys::send(client_ep, &msg);

// Later: revoke the client's capability (and all its children)
sys::cap_revoke(ro_cap)
    .expect("cap_revoke failed");
// ro_cap and any caps derived from it are now invalid
```

Listing 5.1: Capability derivation and revocation.

Chapter 6

IPC Mechanisms

6.1 Synchronous Endpoints

Synchronous IPC follows the L4 rendezvous model: the sender blocks until a receiver is ready, and vice versa. This provides zero-copy message transfer with a context switch directly from sender to receiver.

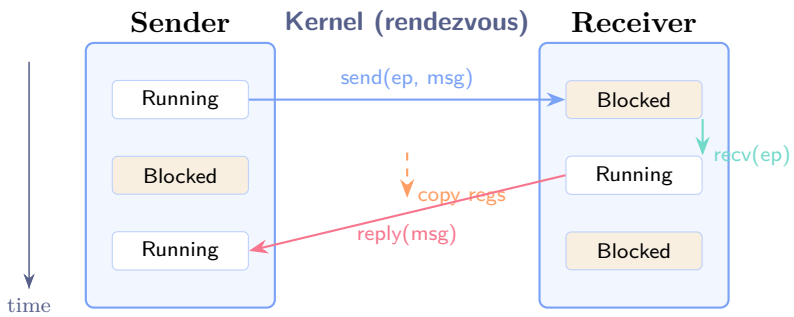


Figure 6.1: Synchronous IPC flow. The sender blocks on `send()`, the receiver blocks on `recv()`, and the kernel performs a direct register-to-register copy at the rendezvous point.

A message consists of up to 8 general-purpose registers (64 bytes total of inline data) plus optional capability transfers in dedicated capability registers. For bulk data, the channel mechanism (Section **??**) is used instead.

6.1.1 IPC Timeout

The `sys::call_timeout(cap, msg, ticks)` interface allows a caller to set a deadline on an IPC call. If the callee does not reply within the specified number of scheduler ticks, the kernel wakes the blocked caller with an error code.

6.2 Asynchronous Channels

Asynchronous channels use lock-free single-producer single-consumer (SPSC) ring buffers for high-throughput, non-blocking data transfer. Each channel is a page-sized ring buffer (`0xA00000`) with separate read and write cursors.

The channel interface provides two operations:

- `chan_send(chan_cap, msg)`: enqueue a message. Returns immediately; fails if the ring is full.

- `chan_recv(chan_cap)`: dequeue a message. Returns immediately; fails if the ring is empty.

6.3 Service Registry

The kernel maintains a name-to-endpoint map that allows services to discover each other:

- `SvcRegister` (syscall 130): register a string name with an endpoint capability.
- `SvcLookup` (syscall 131): look up a name and receive a derived endpoint capability.

6.4 IPC Data Layout

IPC messages carry data in registers with the following layout constraints:

- **Send**: up to 40 bytes of inline data (registers 3–7, 5 registers at 8 bytes each).
- **Receive**: up to 64 bytes (registers 0–7, 8 registers at 8 bytes each), with the byte count in the tag field.
- For larger transfers, the LUCAS layer uses a multi-chunk IPC loop that reassembles data 64 bytes at a time (used for TCP/HTTP).

Chapter 7

Scheduler

7.1 Multi-Level Queue Design

The scheduler implements a four-level priority MLQ with the following queues:

Table 7.1: Scheduler priority levels.

Level	Name	Use Case	Policy
0	Realtime	IRQ handlers, timer ISR	Fixed priority, no preemption
1	High	VMM, FS server	Round-robin, short quantum
2	Normal	Applications	Round-robin, standard quantum
3	Low	Background tasks	Best-effort, long quantum

7.2 Work Stealing

Each CPU maintains a Chase-Lev work-stealing deque (`kernel/src/sched/wsdeque.rs`). When a CPU's local run queue is empty, it attempts to steal threads from other CPUs' deques. This provides automatic load balancing across cores without centralized coordination.

The Chase-Lev deque provides:

- O(1) local push/pop (LIFO for cache locality).
- O(1) remote steal (FIFO for fairness).
- Lock-free operation using atomic compare-and-swap.

7.3 Resource Limits

Each process has configurable resource limits enforced by the scheduler and frame allocator:

- **CPU ticks**: maximum number of scheduler ticks before the process is descheduled. Enforced via `SYS_RESOURCE_LIMIT` (syscall 141).
- **Memory pages**: maximum number of physical frames a process may allocate. Enforced in the frame allocator.

7.4 SMP Support

The kernel supports up to 16 CPUs. Each Application Processor (AP) is woken via the standard INIT-SIPI-SIPI IPI sequence. Key SMP details:

- Each CPU has its own LAPIC timer for preemption.
- Cross-core wakeup uses IPIs (Inter-Processor Interrupts).
- Per-CPU data is accessed via a per-CPU segment or the LAPIC ID.
- SSE must be enabled on *all* CPUs (BSP and APs), as musl and Rust binaries use SSE instructions (e.g., `movaps`).
- The LAPIC timer is calibrated using the PIT (Programmable Interval Timer) with a CPUID.15H/TSC fallback chain for hypervisors.

Chapter 8

Security

8.1 Threat Model

`sotX` defends against:

- Compromised userspace services (no ambient authority).
- Capability confusion attacks (generation counters).
- Stack-based exploits (guard pages, canaries, W^X).
- Address space layout attacks (ASLR).
- Adversarial reconnaissance (deception engine, Chapter **??**).

8.2 W^X Enforcement

The kernel enforces Write XOR Execute at the page table level. No page may have both the writable (W) and executable (X) bits set simultaneously. The `SYS_PROTECT` syscall rejects any request that violates this invariant.

8.3 Stack Security

Three mechanisms protect stacks:

Guard pages. An unmapped page below each stack triggers #PF on overflow.

Stack canaries. A fixed compile-time sentinel value is placed between the return address and local variables. The kernel checks this value on context switch.

ASLR. Stack base addresses are randomized within a configurable range.

8.4 Signed Boot Chain

Every binary in the initrd CPIO archive is Ed25519-signed. The public key is compiled into the init service via `services/init/src/sigkey_generated.rs`. At boot, each binary's SHA-256 hash is verified against its Ed25519 signature before loading. Public-key pinning rejects swap-with-self-signed attempts.

8.5 Provenance Tracking

The SOT provenance subsystem maintains a SHA-256 hash-linked DAG of all object mutations. Each CPU has a dedicated 4096-entry SPSC ring buffer that records provenance events. The hash chain ensures that any tampering with the provenance log is detectable.

Part III

Personalities

Chapter 9

LUCAS: Linux ABI Compatibility

9.1 Architecture

LUCAS (Linux User-mode Compatibility And Syscall translation) implements a comprehensive Linux system call interface on top of the `sotX` kernel. Rather than emulating a Linux kernel, LUCAS translates Linux syscalls into sequences of `sotX` IPC calls and local operations within the init service's `child_handler`.

The key architectural insight is that the kernel intercepts `arch_prctl` at the syscall level (for TLS/FS_BASE management) and redirects all other Linux syscalls via IPC to the init service, which handles them in the context of the calling process.

9.2 Supported Syscalls

LUCAS implements 127+ Linux syscall match arms in the `child_handler` function. Table **??** lists the major categories.

Table 9.1: LUCAS Linux syscall categories.

Category	Syscalls
File I/O	open, openat, read, write, close, lseek, pread64, pwrite64, readv, writev, preadv, pwritev, dup, dup2, dup3, fcntl
Directory	mkdir, mkdirat, rmdir, unlink, unlinkat, rename, renameat, renameat2, getdents64
File metadata	stat, fstat, fstatat, lstat, statfs, fstatfs, access, faccessat, faccessat2, readlink, readlinkat
Process	fork (clone CoW), execve, exit, exit_group, wait4, waitid, getpid, gettid, getppid, clone (CLONE_THREAD)
Memory	brk, mmap, munmap, mprotect, mremap, madvise
Signals	rt_sigaction, rt_sigprocmask, rt_sigreturn, rt_sigpending, rt_sigsuspend, rt_sigtimedwait, tgkill, sigaltstack
Sockets	socket, bind, connect, send, recv, sendto, recvfrom, recvmsg, poll, epoll_create1, epoll_ctl, epoll_wait, socketpair

Continued on next page

Category	Syscalls
Threading	clone (CLONE_VM/FILES/SIGHAND/THREAD), futex, set_tid_address, gettid
Time	clock_gettime, clock_getres, clock_nanosleep, gettimeofday, time, nanosleep
Identity	getuid, getgid, geteuid, getegid, getresuid, getresgid, setuid, setgid, getgroups, setgroups
IPC	pipe, pipe2, eventfd, timerfd_create, timerfd_settime, signalfd, shmget, shmat, shmctl
System	uname, sysinfo, prctl, arch_prctl, personality, getrandom, prlimit64, getrusage
File ops	ftruncate, fsync, fdatasync, fallocate, flock
Special	ioctl, select, pselect6, memfd_create, inotify_init1

9.3 Process Model

LUCAS supports the full Unix process lifecycle:

9.3.1 Fork

`fork()` is implemented via `SYS_AS_CLONE` (syscall 125), which creates a real copy-on-write address space clone. Three critical bugs were fixed in the fork implementation:

1. **Stack save/restore**: The parent's 4 KiB stack at `fork_rsp` is saved to a static buffer (`FORK_STACK_BUF`) before entering the fork-child loop, and restored after exec/exit. This prevents the vfork child from corrupting the parent's stack.
2. **FD state restoration**: Parent FD state is always restored after the fork-child phase completes.
3. **Pipe reference counting**: `PIPE_WRITE_REFS` and `PIPE_READ_REFS` arrays prevent premature EOF from child close.

9.3.2 Exec

`execve()` loads ELF binaries from either the initrd CPIO archive or the VFS. The `exec_loaded_elf()` function in `services/init/src/exec.rs` is the shared ELF loader that handles both static and dynamically-linked binaries:

- Static musl binaries: loaded directly at their ELF load address.
- Dynamic binaries: the PT_INTERP segment is read to find the interpreter (e.g., `ld-musl-x86_64.so.1`), which is loaded at `INTERP_LOAD_BASE` (`0x6000000`). The auxiliary vector includes `AT_BASE` so the linker knows its own load address.

9.3.3 Thread Groups

LUCAS implements thread groups for multi-threaded applications:

- `PROC_TGID`: maps threads to their thread group leader.
- `PROC_FD_GROUP`: threads in the same group share file descriptors.
- `PROC_MEM_GROUP`: threads share the address space.
- `PROC_SIG_GROUP`: threads share signal handlers.

`clone()` with `CLONE_THREAD | CLONE_VM | CLONE_FILES | CLONE_SIGHAND` creates a new thread in the same group, using a clone trampoline that sets up TLS via `arch_prctl`.

9.4 vDSO

The virtual Dynamic Shared Object (vDSO) is a minimal ELF shared library (`ET_DYN`) injected as a single 4 KiB page at `0xB80000`. It exports:

- `__vdso_clock_gettime`: RDTSC-based time, assuming 2 GHz.
- Stubs for `gettimeofday`, `time`, `getcpu`.
- Fork-restore trampoline at `0xB80140`.
- Signal trampoline at `0xB80700`.
- Signal return stub at `0xB80710`.
- Exit stub at `0xB80730`.

The vDSO includes a full ELF structure: `PT_LOAD`, `PT_DYNAMIC`, `.dynsym`, `.dynstr`, SysV `.hash`, `.gnu.version`, and `.gnu.version_d` with the `LINUX_2.6` version tag. `AT_SYSINFO_EHDR` (auxv type 33) points every child process to the vDSO.

9.5 /proc Emulation

`child_handler` intercepts `openat` calls to `/proc/` and `/sys/` paths, returning virtual file descriptors (kind=15) whose contents are generated on-the-fly. Supported paths include:

- `/proc/self/maps`
- `/proc/self/status`
- `/proc/self/stat`
- `/proc/self/auxv`
- `/proc/self/environ`
- `/proc/self/exe` (readlinkat)
- `/proc/cpuinfo`
- `/proc/meminfo`
- `/proc/N/status`
- `/sys/devices/system/cpu/online`

Chapter 10

BSD and rump Kernel Personality

10.1 rump Kernel Integration

The BSD personality uses a real NetBSD rump kernel linked into a freestanding `sotX` service. The rump kernel provides a complete NetBSD kernel environment including VFS, device drivers, and protocol stacks, running entirely in userspace.

The integration comprises:

- **Libraries:** `librump.a`, `librumpvfs.a`, `librumpfs_ffs.a`, `librumpdev.a`, `librumpdev_disk.a` (approximately 2 MiB total).
- **Hypercalls:** 60 hand-rolled `rumpuser_*` functions in `rumpuser_sot.c` that bridge the rump kernel to `sotX` IPC and memory management.
- **Block device:** Connected to the virtio-blk or NVMe driver via IPC for persistent storage.

10.2 Illumos Heirlooms

The v0.2.0+ release includes three Illumos-inspired subsystems:

10.2.1 SMF: Service Management Facility

SMF provides dependency-tracked service management with automatic respawn:

- Services declare dependencies as a topological DAG.
- The SMF manager starts services in dependency order.
- Failed services are automatically restarted via `SYS_THREAD_NOTIFY` (syscall 143).
- Cascade failures propagate through the dependency graph.

10.2.2 Crossbow: Virtual Networking

Project Crossbow provides:

- O(1) VNIC provisioning and revocation.
- Per-port token-bucket rate limiting.
- Locally-administered MAC address generation.
- Virtual switch with multiple forwarding modes.

10.2.3 FMA: Fault Management Architecture

The Fault Management Architecture provides predictive self-healing across five scenarios:

1. Disk retry threshold exceeded.
2. ECC correctable memory errors above threshold.
3. ECC uncorrectable memory errors (immediate response).
4. Thermal events (CPU throttling or shutdown).
5. PCI link degradation (lane width reduction).

Six prioritized correlation rules process events from disk, memory, thermal, and PCI subsystems, combined with provenance anomaly counters.

Chapter 11

Deception Engine

11.1 Capability Interposition Pipeline

The deception engine operates through capability interposition: when a syscall traverses a capability marked with a deception policy, the engine transforms the request or response according to the active deception profile.

The pipeline processes four stages:

1. **Intercept**: The capability's interposition policy (Section **??**) triggers the deception handler.
2. **Profile lookup**: The active deception profile determines what fabrication to apply.
3. **Transform**: The response is rewritten according to the profile's rules.
4. **Consistency check**: The consistency engine verifies cross-channel coherence before delivering the result.

11.2 Deception Profiles

The engine includes five built-in profiles:

Table 11.1: Built-in deception profiles.

Profile	Description
Ubuntu 22.04	Emulates a standard Ubuntu webserver with Apache
Debian	Mimics Debian stable with default packages
CentOS	Presents as a CentOS/RHEL database server
FreeBSD	Emulates a FreeBSD server
Alpine	Lightweight Alpine Linux container

Each profile defines consistent responses for:

- `uname` syscall (kernel version, hostname, architecture).
- `/proc/version` and `/proc/cpuinfo` contents.
- `/etc/os-release` and `/etc/hostname`.
- File system layout and installed packages.

- Network service banners (SSH, HTTP, etc.).

11.3 Live Migration

The deception engine supports live profile migration with the following guarantees:

- **Atomic CSpace swap**: The entire capability space is swapped atomically, ensuring no inconsistent intermediate state.
- **Sub-10 μs latency**: The swap is performed in under 10 microseconds, making it undetectable by timing analysis.
- **Connection preservation**: Active network connections are maintained across profile switches.

11.4 Consistency Engine

The consistency engine ensures cross-channel coherence. When the deception profile claims to be "Ubuntu 22.04", all channels must agree:

- `uname -r` returns the correct kernel version.
- `/etc/os-release` matches.
- `/proc/version` is consistent.
- Network banners use appropriate version strings.
- File timestamps and package lists are plausible.

11.5 Detection Resistance

To resist timing-based detection, the engine employs:

- Calibrated delays matching the expected profile's syscall latency.
- Random jitter added to response times.
- Cache-line isolation to prevent microarchitectural side channels.

Part IV

Services & Drivers

Chapter 12

sotFS: The Filesystem

12.1 Overview

`sotFS` is the native filesystem for `sotX`, designed around a type graph model with formal verification guarantees. The implementation consists of 7 Rust crates, 76 tests, and a mountable FUSE prototype for Linux development.

12.2 Type Graph Model

The filesystem state is represented as a directed graph $TG = (V, E)$ where nodes and edges are typed. The type graph has 6 node types and 6 edge types.

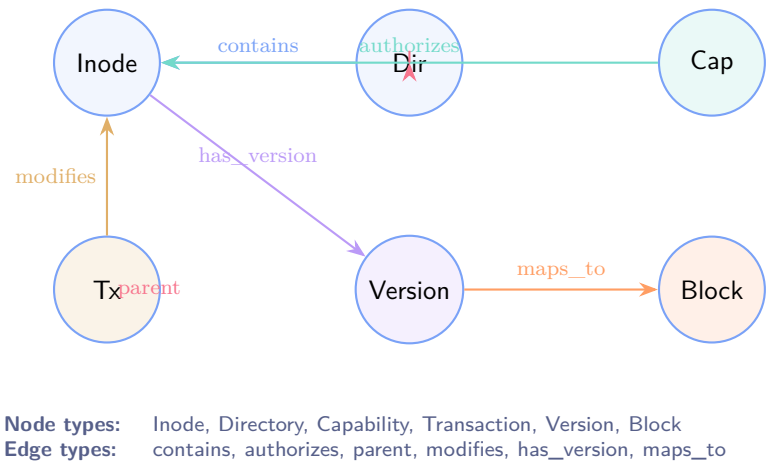


Figure 12.1: sotFS type graph with 6 node types and 6 edge types. Edges show the relationships between filesystem entities.

12.3 DPO Rewriting Rules

POSIX filesystem operations are modeled as Double Pushout (DPO) graph rewriting rules. Each rule specifies a left-hand side (pattern to match), a right-hand side (replacement), and an interface (preserved context).

Definition 12.1 (DPO Rule for CREATE)**.** Given a directory node d and a fresh inode i, the CREATE operation is the DPO rule:

$$L \xleftarrow{l} K \xrightarrow{r} R$$

where:

- $L = \{d : \text{Dir}\}$ — the left-hand side contains only the parent directory.
- $K = \{d : \text{Dir}\}$ — the interface preserves the directory.
- $R = \{d : \text{Dir},\ i : \text{Inode},\ (d \xrightarrow{\text{contains}} i)\}$ — the right-hand side adds the new inode and a "contains" edge.

The rule is applied in context G (the current filesystem state) to produce H (the new state): $G \Rightarrow_{\text{CREATE}} H$.

Table **??** lists all DPO rules.

Table 12.1: POSIX operations as DPO rewriting rules.

VfsOp	**DPO Rule Description**
CREATE	Add Inode node, add "contains" edge from Dir
MKDIR	Add Dir node, add "contains" and "parent" edges
RMDIR	Remove Dir node (requires empty), remove edges
LINK	Add "contains" edge from target Dir to existing Inode
UNLINK	Remove "contains" edge (remove Inode if link count = 0)
RENAME	Remove "contains" from source Dir, add to target Dir
WRITE	Add Version node, add "has_version" edge, add Block nodes

12.4 Bounded Treewidth

Theorem 12.1 (Bounded Treewidth)**.** The treewidth of the sotFS type graph TG satisfies:

$$\text{tw}(TG) \leq \text{LINK_MAX} + O(1)$$

Without hard links, the bound tightens to $\text{tw}(TG) \leq 6$.

This bounded treewidth guarantee ensures that graph algorithms on the filesystem state (such as anomaly detection) run in linear time.

12.5 Ollivier-Ricci Curvature

`sotFS` uses Ollivier-Ricci curvature as an anomaly detection metric. The discrete curvature of an edge (u, v) measures how well-connected the neighborhoods of u and v are. Anomalous filesystem activity (e.g., ransomware creating many files in a flat directory) produces edges with strongly negative curvature, triggering alerts.

12.6 Deceptive Subgraph Projections

The deception functor $D_{FS} : \mathbf{TG} \to \mathbf{TG}$ produces a projected view of the filesystem for deception targets. This functor can:

- Hide real files while presenting fabricated directory listings.
- Inject honeypot files that trigger alerts when accessed.
- Present a plausible filesystem layout matching the active deception profile.

12.7 Implementation Quality

The `sotFS` implementation has been verified through:

- **4 TLA+ specifications** checked with TLC.
- **Typestate enforcement** at compile time.
- **loom concurrency tests** for lock-free data structures.
- **proptest**: property-based testing that found 2 bugs.
- **cargo-fuzz**: 300K runs with 0 crashes.
- **criterion**: performance benchmarks for all operations.
- **76 unit and integration tests**.
- **FUSE prototype**: mountable on Linux for real-world testing.

Chapter 13

Device Drivers

13.1 Driver Architecture

All device drivers run in Ring 3 as capability-holding services. The kernel provides IRQ virtualization: hardware interrupts are demultiplexed and delivered as IPC messages to registered handlers. This architecture ensures that a buggy or compromised driver cannot crash the kernel.

13.2 PCI Enumeration

The `sotos-pci` library provides PCI bus enumeration. It scans the configuration space to discover devices and their BARs (Base Address Registers), then maps MMIO regions into the driver's address space with UC (Uncacheable) flags.

13.3 virtio Drivers

13.3.1 virtio-blk

The virtio-blk driver provides block device access through virtio's split virtqueue mechanism. It handles read and write requests from the filesystem and other services via IPC.

13.3.2 virtio-net

The virtio-net driver provides network interface access. It integrates with the networking stack (Chapter **??**) to send and receive Ethernet frames.

13.4 NVMe Driver

The NVMe driver (`sotos-nvme`) communicates with NVMe SSDs using:

- Admin and I/O submission/completion queue pairs.
- PRP (Physical Region Page) lists for scatter-gather DMA.
- IPC-based interface for filesystem requests.

13.5 xHCI USB Driver

The xHCI driver (`sotos-xhci`) manages USB 3.x host controllers:

- Device slot allocation and endpoint configuration.
- Transfer ring management (command, event, transfer rings).
- HID keyboard support: USB keyboard events are translated and delivered to the keyboard ring buffer.

13.6 PS/2 Keyboard and Mouse

The PS/2 driver handles legacy keyboard and mouse input:

- Keyboard: scan code set 1/2 translation, key-up/key-down events written to the ring buffer at `0x510000`.
- Mouse: PS/2 mouse protocol with movement and button events written to the ring buffer at `0x520000`.

13.7 AC97 Audio

The AC97 audio driver provides basic audio output through the AC97 codec interface, supporting PCM playback at standard sample rates.

13.8 Minimal Service Skeleton

Listing **??** shows the minimal code to create a new service that registers with the service registry and enters an IPC loop.

```rust
#![no_std]
#![no_main]

use sotos_common::sys;
use sotos_common::ipc::Message;

#[no_mangle]
pub extern "C" fn _start() -> ! {
    // Read our capabilities from BootInfo
    let boot_info = unsafe { &*(0xB00000 as *const BootInfo) };
    let root_cap = boot_info.root_cap;

    // Create an endpoint for receiving requests
    let ep = sys::endpoint_create()
        .expect("endpoint_create failed");

    // Register our service name so others can find us
    sys::svc_register("my_service", ep)
        .expect("svc_register failed");

    // Main IPC loop: receive requests and send replies
    loop {
        let mut request = Message::new();
        let badge = sys::recv(ep, &mut request);

        // Dispatch based on the message tag
        let mut reply = Message::new();
        match request.tag() {
            1 => { /* handle command 1 */ }
            2 => { /* handle command 2 */ }
            _ => { reply.set_error(-1); }
        }
```

```

        sys::reply(badge, &reply);
    }
}
```

Listing 13.1: Minimal service skeleton.

Chapter 14

Networking Stack

14.1 Protocol Stack

The networking stack is implemented as a userspace service with the following layers:

Layer 2: Ethernet. Frame transmission and reception via virtio-net driver.

ARP. Address Resolution Protocol with dynamic cache.

DHCP. Dynamic host configuration with random transaction IDs.

Layer 3: IP. IPv4 packet routing and fragmentation.

ICMP. Ping echo request/reply and error messages.

Layer 4: UDP. Connectionless datagram transport. Used for DNS.

Layer 4: TCP. Connection-oriented transport with:

- Three-way handshake (SYN, SYN-ACK, ACK).
- Retransmission with exponential backoff.
- RST handling for connection reset.
- TCP Reno congestion control.
- 16 KiB receive buffer.

DNS. Domain name resolution via UDP to configured resolver (default: `10.0.2.3` under QEMU SLIRP).

14.2 Socket API

LUCAS exposes a BSD socket API to Linux binaries:

- `socket(AF_INET, SOCK_STREAM|SOCK_DGRAM, 0)`: creates TCP or UDP socket FDs.
- `bind`, `connect`, `listen`, `accept`.
- `send`, `recv`, `sendto`, `recvfrom`, `recvmsg`.
- `poll`, `epoll_create1`, `epoll_ctl`, `epoll_wait`.

The socket proxy translates these into IPC messages to the net service:

- TCP send: `regs=[conn_id, 0, data_len, data...]` (data in regs[3..], 40 bytes max per message).
- UDP sendto: `regs=[dst_ip, dst_port, src_port, data_len, data...]` (data in regs[4..]).
- Receive: reply tag = byte count, data in regs[0..7] (64 bytes max).

14.3 HTTP Client

The networking stack supports HTTP/1.1 GET requests sufficient for tools like `busybox wget`. Multi-chunk IPC assembly handles responses larger than 64 bytes by looping the receive call.

Part V

Operations

Chapter 15

Building and Running

15.1 Prerequisites

- Rust nightly toolchain (for `build-std` and `abi_x86_interrupt`).
- `x86_64-unknown-none` target: `rustup target add x86_64-unknown-none`.
- Python 3 (invoked as `python` on Windows).
- QEMU (at `C:\Program Files\qemu\` on Windows).
- `just` command runner.

15.2 Build Commands

Table 15.1: Build and run commands.

Command	Description
`just build`	Compile kernel and all services
`just run`	Build + create disk image + boot QEMU (serial)
`just run-gui`	Same as `run` but with QEMU display window
`just run-net`	Boot with virtio-net enabled
`just run-nvme`	Boot with NVMe device
`just run-xhci`	Boot with xHCI USB controller
`just run-fast`	Boot with WHPX acceleration (~25s to Tier 5)
`just debug`	Boot with GDB server (`target remote :1234`)

15.3 Manual Build Steps

```
# Compile the kernel
cargo build --package sotos-kernel

# Create the disk image (GPT + FAT32 with LFN)
python scripts/mkimage.py

# Boot in QEMU (2048M RAM, serial to terminal)
"C:/Program Files/qemu/qemu-system-x86_64.exe" \
  -drive format=raw,file=target/sotx.img \
  -serial stdio -display none -no-reboot -m 2048M
```

Listing 15.1: Manual build and boot sequence.

15.4 Disk Image

The disk image is built by `scripts/mkimage.py`, a pure-Python GPT + FAT32 image builder with Long File Name (LFN) support. It creates a raw disk image containing:

- A GPT partition table with an EFI System Partition.
- The Limine bootloader (BIOS stage 1 and 2).
- The kernel ELF binary.
- The initrd CPIO archive containing all services.
- The `limine.conf` boot configuration.

Chapter 16

Debugging

16.1 Serial Output

All kernel debug output goes through COM1 serial (`kprintln!()` macro). QEMU's `-serial stdio` flag redirects this to the terminal. The `kdebug!()` macro is gated behind the `verbose` feature flag, allowing conditional debug output.

Critical invariant: Serial must be initialized *first* in `kmain()`, before any assertions or debug prints. A panic before serial initialization produces no diagnostics.

16.2 GDB Debugging

```
# Terminal 1: boot with GDB server
just debug

# Terminal 2: connect GDB
gdb -ex "target remote :1234"
(gdb) break kmain
(gdb) continue
```

Listing 16.1: GDB debugging session.

16.3 Common Gotchas

ET_DYN rejection. Limine rejects dynamically-linked ELF files. Ensure `-C relocation-model=static` is set.

Cargo config merging. Child crate builds inherit parent `.cargo/config.toml`. Use `CARGO_ENCODED_RUSTFLA` to override.

TCG spin deadlock. Under QEMU TCG (software emulation), tight spin loops may deadlock. Use `yield` or `hlt`.

WHPX strict mode. WHPX (Windows Hypervisor Platform) enforces strict x86 semantics (FXRSTOR reserved bits, MXCSR validation) that TCG ignores.

IST and #PF. Never assign an IST entry to #PF—shared IST stacks corrupt under concurrent CoW faults.

Init stack size. Use 4 pages (16 KiB), not 1. Smaller stacks overflow during service initialization.

MMIO UC flags. Device MMIO pages must be mapped with Uncacheable flags to prevent CPU write-back caching.

compiler_builtins memcmp. In debug builds, `compiler_builtins` `memcmp` triggers UB checks. Use byte-by-byte comparison in `kernel/src/initrd.rs`.

Atomic thread redirect. Use `spawn_user_with_redirect()` —setting redirect_ep after thread creation races with early syscalls.

Chapter 17

Syscall Reference

17.1 Kernel Syscalls

Table ?? lists all kernel-native syscalls.

Table 17.1: Kernel syscall reference.

#	Name	Arguments	Returns / Description
24	UnmapFree	vaddr	Unmap page + free frame
130	SvcRegister	name, ep_cap	Register service name
131	SvcLookup	name	Returns derived ep cap
132	InitrdRead	filename, buf	Read file from initrd CPIO
133	BootInfoWrite	as_cap, addr	Write BootInfo into target AS
134	Protect	addr, len, prot	mprotect with W^X
135	CallTimeout	cap, msg, ticks	IPC call with timeout
140	ThreadInfo		Get tid/priority/ticks/mem
141	ResourceLimit	ticks, pages	Set CPU/memory limits
142	ThreadCount		Get total thread count
143	ThreadNotify	tid	Wake thread (for SMF)
160	SetFsBase	addr	Set FS_BASE MSR (TLS)
161	GetFsBase		Get FS_BASE MSR
172	GetThreadRegs	raw_tid	Read saved registers
173	SignalEntry	thread_cap, addr	Set signal trampoline
174	SignalInject	raw_tid, signum	Set pending signal bit
252	DebugFreeFrames		Return free frame count

17.2 SOT Syscalls

See Table ?? in Chapter ?? for the 12 SOT exokernel syscalls (numbers 300–311).

17.3 FD Kind Codes

Table ?? lists the file descriptor kind codes used by LUCAS to dispatch operations to the correct handler.

Table 17.2: File descriptor kind codes.

Kind	**Name**	**Description**
1	Stdin	Standard input (keyboard ring)
2	Stdout	Standard output (serial)
3	Stderr	Standard error (serial)
8	Null	`/dev/null`
9	Zero	`/dev/zero`
10	PipeRead	Read end of a pipe
11	PipeWrite	Write end of a pipe
13	VfsFile	VFS-backed regular file
14	VfsDir	VFS-backed directory
15	ProcFs	Virtual /proc or /sys file
16	TcpSocket	TCP socket (via net IPC)
17	UdpSocket	UDP socket (via net IPC)
18	Initrd	File backed by initrd CPIO
22	EventFd	eventfd counter
23	TimerFd	timerfd (TSC-based)
25	MemFd	memfd (anonymous page-backed)
27	UnixA	socketpair end A
28	UnixB	socketpair end B

Chapter 18

Formal Verification

18.1 TLA+ Specifications

Six TLA+ specifications cover the core kernel subsystems. Each has a matching TLC model-checking configuration file.

Table 18.1: TLA+ specifications and their coverage.

Specification	Coverage
capabilities	CDT derivation, revocation, rights attenuation
ipc	Endpoint send/recv rendezvous, deadlock freedom
scheduler	MLQ priority inversion freedom, work stealing
sot_capabilities	SOT layer cap derive/revoke with interposition
sot_provenance	Hash-chain integrity, ring buffer overflow
sot_transactions	Tier 0/1/2 commit/abort, serialization

All specifications are checked via `just tlc-mc`, which runs TLC on each spec and reports per-specification state counts.

18.2 sotFS Formal Methods

The `sotFS` filesystem uses multiple formal verification approaches:

DPO Rewriting. Each POSIX operation is modeled as a Double Pushout rule on the type graph (see Section **??**).

Bounded Treewidth. Proved that $\mathrm{tw}(TG) \leq 6$ without hard links, ensuring linear-time graph algorithms.

Typestate. Rust's type system enforces valid state transitions at compile time (e.g., a transaction handle cannot be used after commit).

loom. Concurrency testing for lock-free data structures.

proptest. Property-based testing (found 2 bugs: an off-by-one in directory iteration and a missing EOF marker).

cargo-fuzz. Coverage-guided fuzzing: 300K runs, 0 crashes.

18.3 Invariants

The following invariants are maintained by the system and checked by the TLA+ specifications:
Invariant 18.1 (Capability Monotonicity)**.** For any capability c derived from p: $\text{rights}(c) \subseteq \text{rights}(p)$.
Invariant 18.2 (IPC Deadlock Freedom)**.** No cycle exists in the endpoint wait graph.
Invariant 18.3 (Transaction Serializability)**.** The commit order of Tier 2 transactions is equivalent to some serial execution.
Invariant 18.4 (Provenance Integrity)**.** The SHA-256 hash chain in the provenance ring is tamper-evident: $h_i = \text{SHA256}(h_{i-1} \| \text{event}_i)$.
Invariant 18.5 (W^X)**.** For all pages p in any address space: $\neg(\text{writable}(p) \wedge \text{executable}(p))$.

Appendix A

Linux ABI Constants

The `sotos-common::linux_abi` module defines 198 `SYS_*` constants for Linux system call numbers. Key structures defined in this module:

Table A.1: Linux ABI structures in sotos-common.

Struct	Size	Description
`Stat`	144 B	File status (matches Linux x86_64)
`Termios`	44 B	Terminal I/O settings
`Winsize`	8 B	Terminal window size
`KernelSigaction`	32 B	Signal action (24 + sigsetsize)
`LinuxDirent64`	var	Directory entry

Important ABI notes:

- `rt_sigaction`: The kernel sigaction struct is 32 bytes (24 + 8-byte sigset), *not* 152. Writing 152 bytes to `oldact` corrupts stack return addresses.
- `TCGETS` ioctl: reads 36 bytes (not 60).
- `fstat` on kind=14 (VfsDir): returns `S_IFDIR`.

Appendix B

Project Structure

`kernel/` The microkernel (Ring 0)

`arch/x86_64/` CPU-specific: serial, GDT, IDT, LAPIC, SMP boot, SYSCALL/SYSRET

`mm/` Physical frame allocator (bitmap), slab allocator

`cap/` Capability table + CDT

`ipc/` Endpoints (sync) + Channels (async)

`sched/` Thread management, MLQ scheduler, Chase-Lev deque

`libs/` Shared libraries

`sotos-common/` Kernel-userspace ABI, syscall numbers, linux_abi module

`sotos-pci/` PCI bus enumeration

`sotos-virtio/` virtio device drivers

`sotos-net/` IP/TCP/UDP/ICMP networking

`sotos-nvme/` NVMe SSD driver

`sotos-xhci/` xHCI USB driver

`sotos-objstore/` Object store + VFS

`sotos-ld/` Dynamic linker

`services/` Userspace servers

`init/` First userspace process (spawns all others)

`vmm/` Virtual memory manager (paging, CoW)

`kbd/` Keyboard driver service

`net/` Network service

`nvme/` NVMe driver service

`xhci/` xHCI USB driver service

`personality/` OS personality layers

`bsd/` NetBSD rump kernel integration

`linux/` Linux compatibility (LUCAS, LKL)

`common/deception/` Deception engine

`boot/` Bootloader configuration (`limine.conf`)

`scripts/` Build tools (`mkimage.py`, test scripts)

`docs/` Documentation and specifications

`specs/` TLA+ formal specifications

Appendix C

VfsOp to DPO Rule Mapping

Table **??** provides the complete mapping from VFS operations to their DPO rewriting rules, including the graph elements affected.

Table C.1: Complete VfsOp to DPO rule mapping.

VfsOp	**L (match)**	**K (keep)**	**R (produce)**	**Gluing**
CREATE	Dir d	Dir d	d, Inode i, $d \to i$	d identity
MKDIR	Dir d	Dir d	d, Dir d', $d \to d'$, $d' \to d$	d identity
RMDIR	Dir d, Dir d', $d \to d'$	Dir d	Dir d	empty d'
LINK	Dir d, Inode i	d, i	d, i, $d \to i$	d, i id
UNLINK	Dir d, $d \to i$	Dir d	Dir d	nlink check
RENAME	Dir s, $s \to i$, Dir t	s, i, t	s, i, t, $t \to i$	s, t id
WRITE	Inode i	Inode i	i, Ver v, Blk b, edges	i identity

Appendix D

Configuration Reference

D.1 QEMU Flags

Table D.1: Common QEMU flags used with sotX.

Flag	Purpose
`-serial stdio`	Redirect COM1 to terminal
`-display none`	Headless (serial-only)
`-no-reboot`	Halt on triple fault
`-m 2048M`	2 GiB RAM
`-smp N`	N CPU cores
`-accel whpx`	Windows Hypervisor (fast)
`-accel tcg`	Software emulation (slow)
`-s`	GDB server on port 1234
`-device virtio-net-pci`	virtio network adapter
`-device nvme`	NVMe SSD controller
`-device qemu-xhci`	xHCI USB 3.0 controller
`-netdev user,id=n0`	SLIRP user-mode networking

D.2 Key Compile Flags

`-C relocation-model=static` Required: Limine rejects `ET_DYN`.

`-Z build-std=core,alloc` Build `core` and `alloc` from source for the bare-metal target.

`-C code-model=kernel` Place code in the upper 2 GiB (optional, for HHDM compatibility).

`CARGO_ENCODED_RUSTFLAGS` Override parent `.cargo/config.toml` for userspace crate builds.

Glossary

AP Application Processor (non-boot CPU in SMP).

BSP Bootstrap Processor (the CPU that runs firmware).

CDT Capability Derivation Tree.

CoW Copy-on-Write.

DPO Double Pushout (graph rewriting framework).

FMA Fault Management Architecture.

HHDM Higher Half Direct Map.

IPC Inter-Process Communication.

IST Interrupt Stack Table.

LAPIC Local Advanced Programmable Interrupt Controller.

LUCAS Linux User-mode Compatibility And Syscall translation.

MLQ Multi-Level Queue (scheduler).

PRP Physical Region Page (NVMe scatter-gather).

SMF Service Management Facility.

SO Secure Object.

SOT Secure Object Transaction.

SPSC Single-Producer Single-Consumer (ring buffer).

TLC TLA+ model checker.

UC Uncacheable (memory type for MMIO).

vDSO virtual Dynamic Shared Object.

VNIC Virtual Network Interface Card.

VFS Virtual File System.

VMM Virtual Memory Manager.

W^X Write XOR Execute.